⚙️ Rule Engine — Implementation Guide

Proposed Solution

- Instead of adding a new attribute hard coded in the code
- We can build the attribute’s condition based on its data type
- As in Drool file , same data types are represented in the same way

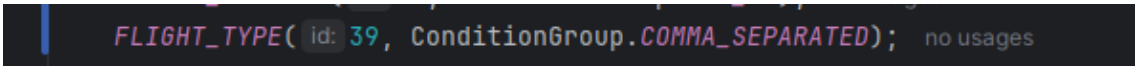
How to add a new attribute condition in DB?

- in TT_ATTR_MAP table , insert the new attribute meta data
  - Like →

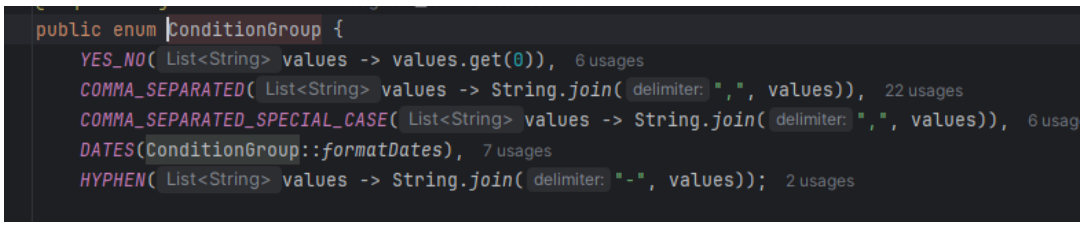
```
INSERT INTO wonderdb.TT_RULES_ATTR_MAP (
    RULES_ATTR_UID,
    RULES_ATTR_CAT_UID,
    ATTR_TYPE,
    ENTITY_NAME,
    ATTRIBUTE_NAME,
    STATUS,
    ATTR_REFERRED_ENTITY,
    REFERRED_ATTRIBUTE,
    REFERRED_OPERATOR,
    REFACTORED_WAY
)
VALUES (
    39, -> NOT AUTO INCREMENT
    1,
    5,
    'RulesFlightBean',
    'flightType',
    1,
```

```
    'EnumList',
    'FLIGHT_TYPE', -> is the enum name (TYPE_NAME) in TT_E
NUM_VALUES
    '0,1',
    1
);
```

- Add the new attribute in RulesFlightBean class
- Set this new attribute in the following functions
  - `processRulesFlightBeanRoundTrip`
  - `processRulesFlightBeanMulticity`
  - `processRulesFlightBean`
  - This to take the value of this attribute from FlightOption object
- In Rule Engine CRUD
  - Add the new attribute in `ConditionAttribute` ENUM



    - FLIGHT_TYPE → name of the new attribute
    - 39 → ATTR_UID (NOT AUTO Increment)
    - ConditionGroup.COMMA_SEPARATED → Choose the separation of the value of this new attribute



Rule Engine Enhancement – Expression Builder Design

Overview

This document describes the **new expression-building approach** for the Rule Engine. The goal is to create a **modular, extensible, and type-safe system** for generating rule conditions dynamically based on:

- Attribute type
- Operator
- Criteria values

The system follows a **strategy-based design**, where each data type has its own expression builder.

Key Components

1. `RuleEngineConditionExpression` (Interface)

Defines the contract for all expression builders.

```
String buildConditionExpression(RuleCriteriaElements ruleCrit
eria,
                                RuleEngineAttributeDTO ruleEn
gineAttributeDTO)
```

Each implementation handles a specific data type or scenario.

Expression Builders

2. `RuleEngineDateExpression`

Handles all **date-related attributes**.

Supported Operations

- Simple comparisons (`=`, `>`, `<`, etc.)
- `BETWEEN`

Behavior

1. Simple Date Expression

Formats input date and builds:

```
( departureDate >= "01-Jan-2024 00:00")
```

2. BETWEEN Operator

Two scenarios:

a. Blackout Dates

- Expands date ranges into **individual days**
- Produces:

```
blackOutStartEndDate not contains("2024-01-01") &&
blackOutStartEndDate not contains("2024-01-02")
```

b. Regular Date Range

- Converts to:

```
( departureDate >= "01-Jan-2024 00:00") &&
( departureDate < "05-Jan-2024 00:00")
```

- `toDate` is incremented by **+1 day** to make the range inclusive.

---

3. `RuleEngineListExpression`

Handles **list-based attributes**.

Supported Operators

- `IN`
- `NOT_IN`

Behavior

IN

```
(attribute.contains("A") || attribute.contains("B"))
```

NOT IN

```
!(attribute.contains("A") && attribute.contains("B"))
```

Key Logic

- Uses `contains()` for all comparisons
- Dynamically handles:
  - String values → `"value"`
  - Numeric values → `value`

---

4. `RuleEngineSingleValExpression`

Handles **single-value attributes**.

Supported Cases

1. Simple Comparison

```
(price >= 100)
(status == "ACTIVE")
```

2. Range Handling (Special Case)

Triggered when:

```
attributeType == 10
```

Input:

```
100-200
```

Output:

```
(price >= 100) && (price <= 200)
```

Supporting Enums

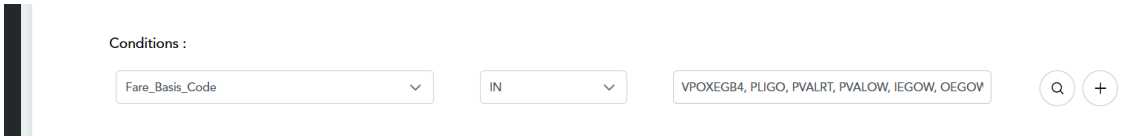
5. `RuleDataTypeEnum`

Maps attribute types to logical data types:

Type	Codes
INTEGER	1
DATE	3
STRING	8
LIST_OF_STRING	2, 4, 5
DOUBLE	6
LIST_OF_INTEGER	7

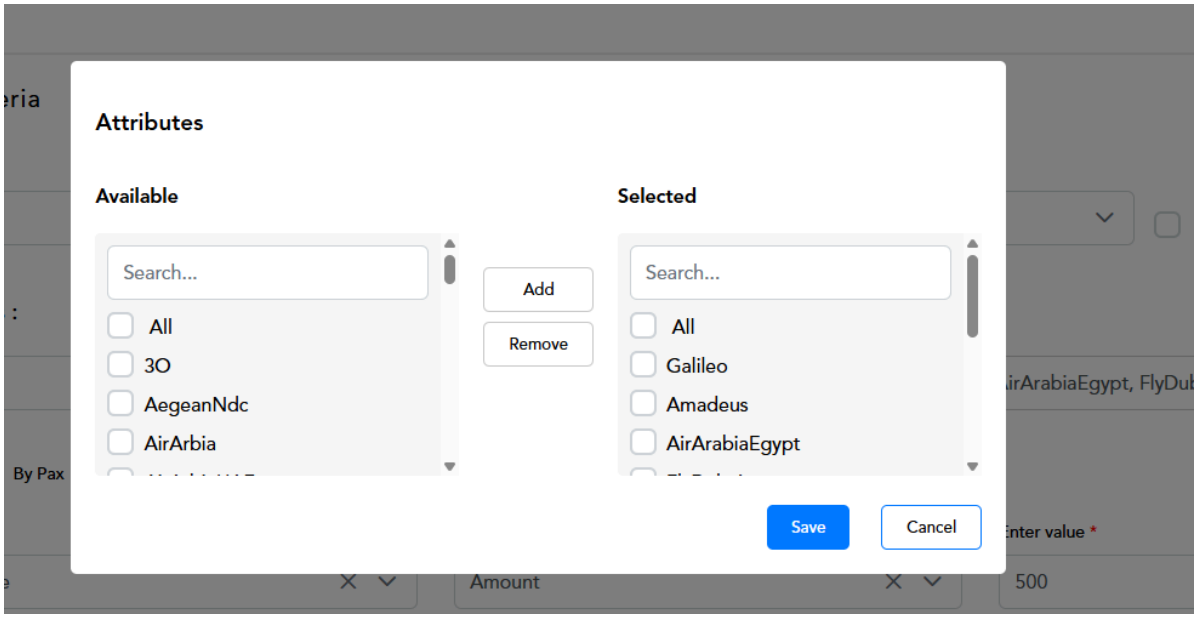
NOTE That For LIST_OF_STRING Type

- 2 → Handles List of string if the user enters them as a text in UI



- 4,5 → Handle List of String if the user selects from a dropdown from UI

- 4 → if these values come from an Entity from DB like TT_TS_SUPPLIER
- 5 → if these values come from an ENUM from TT_ENUM_VALUES table



Provides:

```
RuleDataTypeEnum.fromCode(int code)
```

---

6. `RuleEngineOperatorEnum`

Maps operator codes to actual operators:

Operator	Code
IN	0
NOT_IN	1
EQUAL	2
BETWEEN	10

Provides:

```
RuleEngineOperatorEnum.fromCode(int code)
```

---

7. `DateAttributes`

Defines attributes treated as **blackout date types**.

```
BLACKOUT_START_END_DATE
BLACKOUT_START_END_DATE_CAR
BLACKOUT_START_END_DATE_ACTIVITY
```

Used to determine:

- Expand to individual dates OR
- Treat as normal range

Design Principles

1. Strategy Pattern

Each data type has its own expression builder:

- Date → `RuleEngineDateExpression`
- List → `RuleEngineListExpression`
- Single Value → `RuleEngineSingleValExpression`

---

2. Separation of Concerns

- Parsing → handled inside each builder
- Formatting → delegated to `CommonUtil`
- Operator mapping → enums

---

3. Extensibility

Adding new data types or operators is easy:

- Add new enum value
- Add new expression builder (if needed)

---

4. Reusability

Shared logic:

- `extractCriteriaValues()`
- Data type checks
- Operator mapping

Execution Flow

1. Fetch rule from DB (`RuleCriteriaElements`)
2. Identify:
   - Attribute name
   - Attribute type
   - Operator
3. Resolve:

```
RuleDataTypeEnum.fromCode(attributeType)
```

4. Select appropriate builder:
   - Date → `RuleEngineDateExpression`
   - List → `RuleEngineListExpression`
   - Otherwise → `RuleEngineSingleValExpression`
5. Generate DRL condition string

Example End-to-End

Input

```
attributeName = departureDate
operator = BETWEEN
```

```
criteriaValue = 01-01-2024,05-01-2024
```

Output

```
( departureDate >= "01-Jan-2024 00:00") &&
( departureDate < "06-Jan-2024 00:00")
```
